

Sustainability of Embedded Systems



Anna Baranek
Isabella Süss
Adrian Ruckstuhl
Severin Steiner

Context

The efficiency of embedded systems is not strictly a hardware concern. Even the most advanced, energy-efficient hardware can be undermined by poorly optimized software. While adding more RAM or faster CPUs might seem like a simple solution to performance issues, this approach is often unsustainable and, in many constrained environments, physically impossible. In applications such as IoT sensors for example, the quality of the code directly dictates battery life, thermal behavior, and overall system reliability.

Motivation

The motivation behind this study stems from the industry-wide shift toward high-level languages and abstractions. As developers move away from low-level assembly, a "conceptual gap" emerges: hardware is often treated as an infinite resource. In reality, however, every CPU cycle and every byte of memory carries a measurable cost in both energy and time.

To investigate the impact of software optimization on energy consumption and runtime, this report focuses on complexity of algorithms and the overhead introduced by I/O operations.

Method

To compare various programming approaches regarding their efficiency, performance, and energy consumption directly on the hardware, specific test series were conducted and measured. As shown in Figure 1 below, the CT Board (featuring an STM32F429/ARM Cortex-M4 microcontroller) was utilized. This device was chosen because it simplifies the power measurements, which are difficult to obtain on conventional computer systems without specialized measuring equipment. Additionally, the microcontroller environment allows execution close to bare-metal, without the influence of a full operating system, allowing for more accurate and reproducible results. The full specifications of the device can be found following the [link](#) in the appendix. Measurements were performed over 10 seconds using a Keysight power analyzer, connected to the regions highlighted in red in Figure 1.

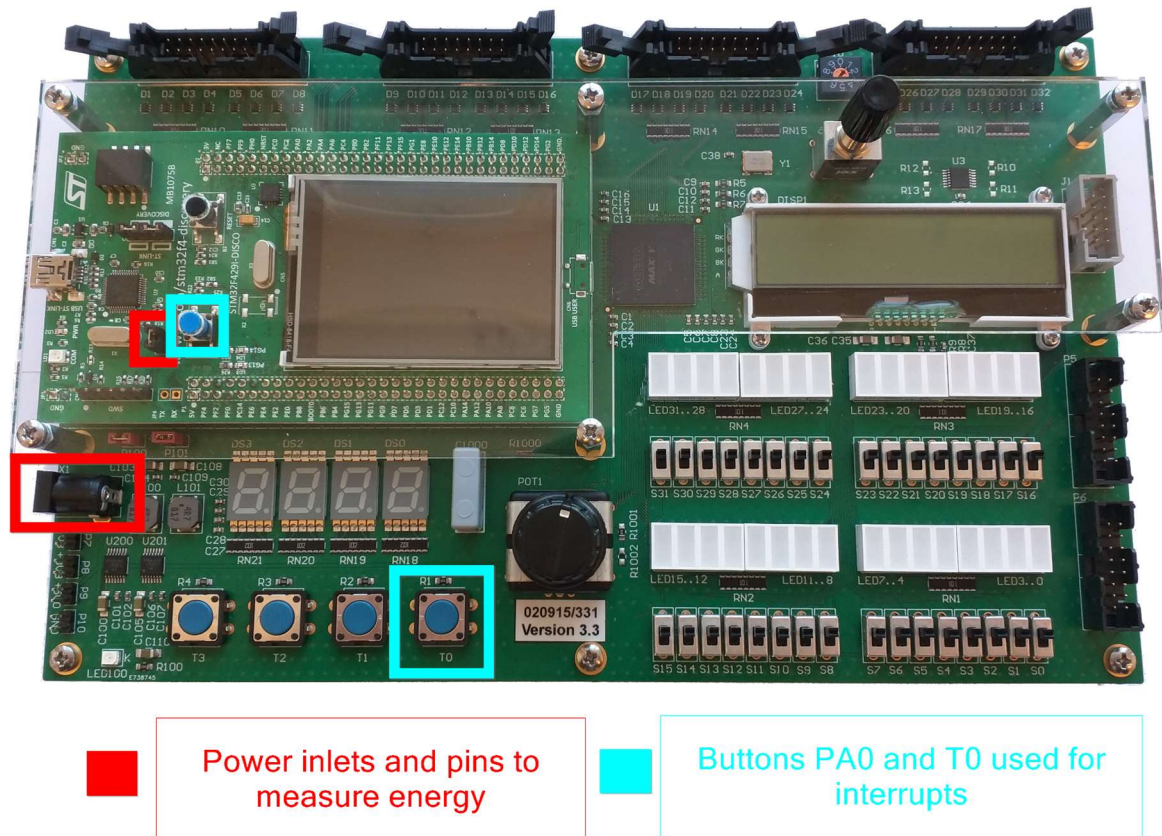


Figure 1: CT Board. [1]

Difficulties

Measuring code optimization proved challenging due to numerous interfering factors. Peripheral components such as LEDs, displays, or status lamps can consume significantly more power than the CPU core itself. In Figure 2, the LEDs were toggled every second within a loop. The average power consumption over 10 seconds was approximately 172 mA, whereas the same measurement without active LEDs dropped into the μA range. This demonstrates that core optimization measurements occur within an extremely small margin, as turning on LEDs increases power consumption by a factor of ~ 200 compared to the baseline results. A general problem during measurements was that the equipment setup was not specifically dedicated to the CT Board, and the measured values were extremely small. Interference and noise were significant issues, even physical contact led to outliers. Consequently, only general trends can be analyzed.

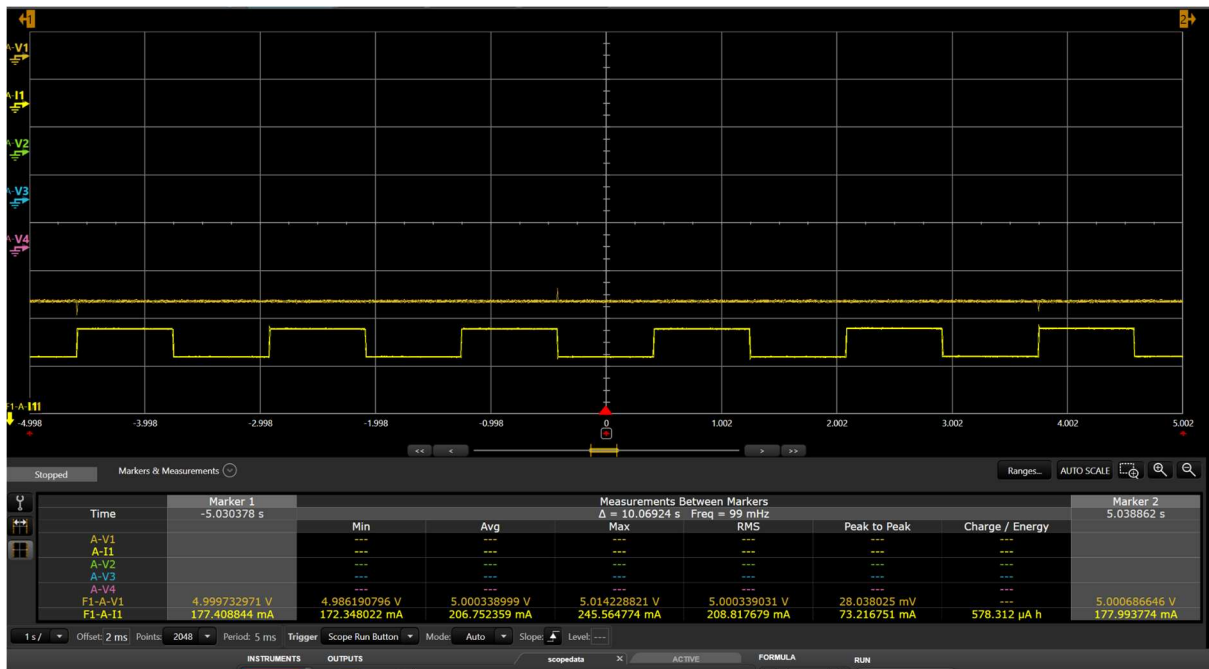


Figure 2: Timer with LED

Results

The following describes the tests performed and presents the corresponding results.

Busy-Wait vs. Interrupt-Control

This experimental section analyzes CPU activity during event processing with three different approaches. The code compares a busy-wait implementation, which polls the input register (BUTTON_T0) in a permanent loop, with interrupt-control via the EXTI unit. Two interrupt states are distinguished: in standard interrupt mode, the CPU remains active while waiting for the event (Idle), whereas in sleep mode, the `__asm volatile("wfi")` Wait-for-Interrupt command is used to physically transition the core into a low-power state until a hardware interrupt triggers reactivation. During each test, the button was pressed four times to trigger the interruption.

An interrupt-driven approach using "Wait-for-Interrupt" (WFI) and especially "Sleep-on-Exit" is expected to show a measurable decrease in power consumption compared to a standard busy-wait loop.



Figure 3: Busy Loop

In the measurement of the busy-wait implementation, four distinct current peaks ("humps") are visible, representing the physical interaction with button T0 (light blue button at the bottom of figure 1). Since GPIO pins require additional energy during a status change, each button press leads to a measurable current flow. A key aspect of the experimental setup is the spatial separation of the components: while the interrupt with and without sleep mode was triggered via the blue button on the upper component of the CT Board, button T0 is located on a separate component, see the blue marking in Figure 1. Since the measurement was performed on the upper part of the board (see the upper red marking in Figure 1), the additional communication to the external button T0 must be considered as a potential influencing factor when interpreting absolute values. Furthermore, it should be noted that the recorded values show a negative current flow, which indicates a significant offset shift in the measurement setup. Because of these non-absolute negative ampere readings, the data was not evaluated based on absolute precision.

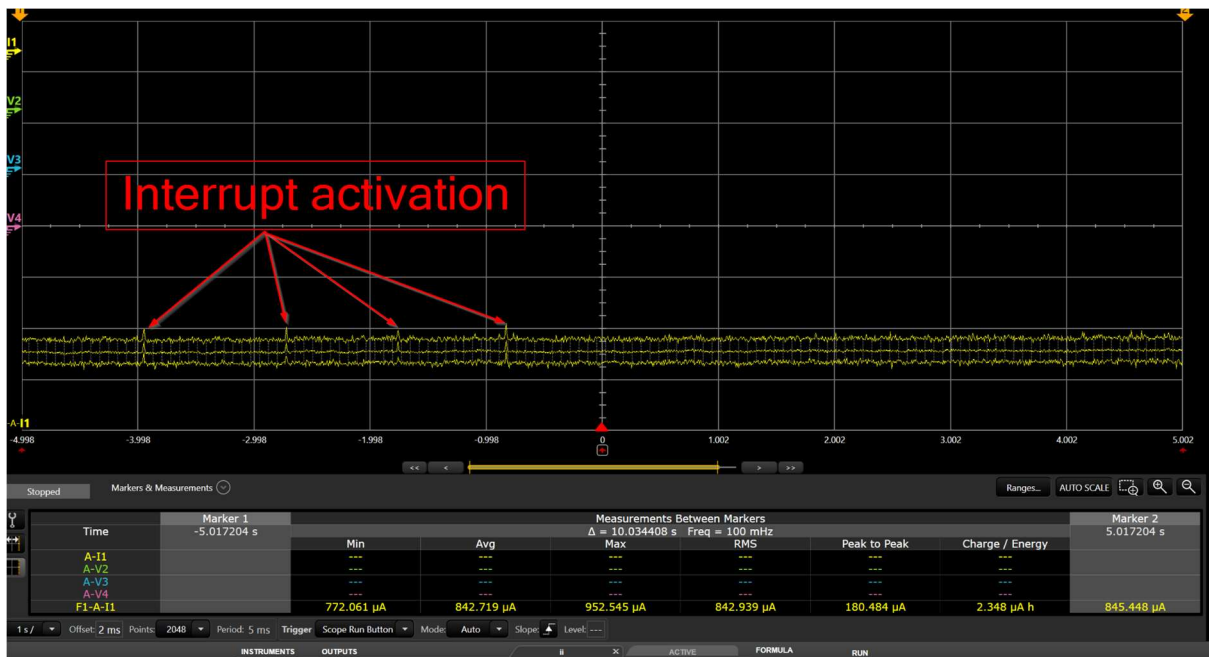


Figure 4: Normal Interrupt

In Figure 4, small spikes are visible, signaling the interruption of the regular program flow. The results show that this mode is significantly less energy-efficient than the sleep mode. Surprisingly, the standard interrupt also performs worse than the simple busy-loop.

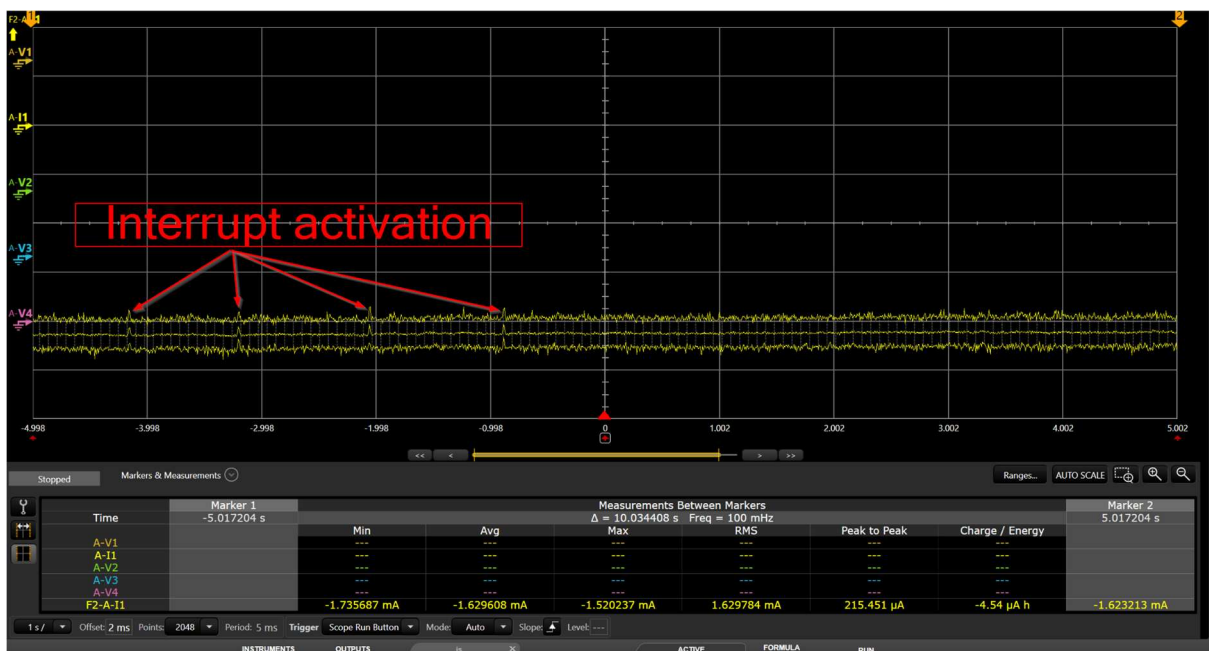


Figure 5: Sleep

As theoretically expected, this test series provides the lowest consumption values of the three scenarios. The four interactions are only slightly indicated in the curve of Figure 5, as the system returns to the power-saving state immediately after each task is processed.

Discussion

The comparison demonstrates that sleep mode is the most efficient solution, with a difference of approximately 2.5 mA compared to the standard interrupt. Interestingly, the standard interrupt proved to be the least favorable variant in this specific setup. It is hypothesized that this is caused by the administrative overhead of context switching (saving registers) combined with the additional energy required by the active EXTI and NVIC hardware logic. In contrast, the busy-loop benefits from high pipeline efficiency. As previously mentioned, the physical localization of the button (communication overhead between board components) may also influence the measured values. For this specific, low-frequency event, the results suggest that interrupt architectures only reach their full sustainability potential when combined with low-power modes ([wfi](#)). However, this outcome depends heavily on the polling frequency and the complexity of the task. Without this optimization, a simple loop can paradoxically be the more energy-efficient choice.

Software-Loop vs. Hardware-Timer

This test case documents the efficiency of two different methods for toggling LEDs at a fixed frequency. Due to the previously explained issue regarding the high power consumption of the LEDs themselves, their physical illumination was commented out in the code. However, the corresponding register accesses remained in place to realistically represent the CPU load. The software variant ties up the CPU through simple decrementing in a blocking loop, whereas the optimized version utilizes the TIM4 hardware timer. In this case, the time base is handled directly by hardware prescalers and auto-reload registers, allowing the CPU to remain idle and only become active during the Interrupt Service Routine (ISR). Consequently, the timer-based approach is expected to be more efficient as it reduces unnecessary CPU cycles.

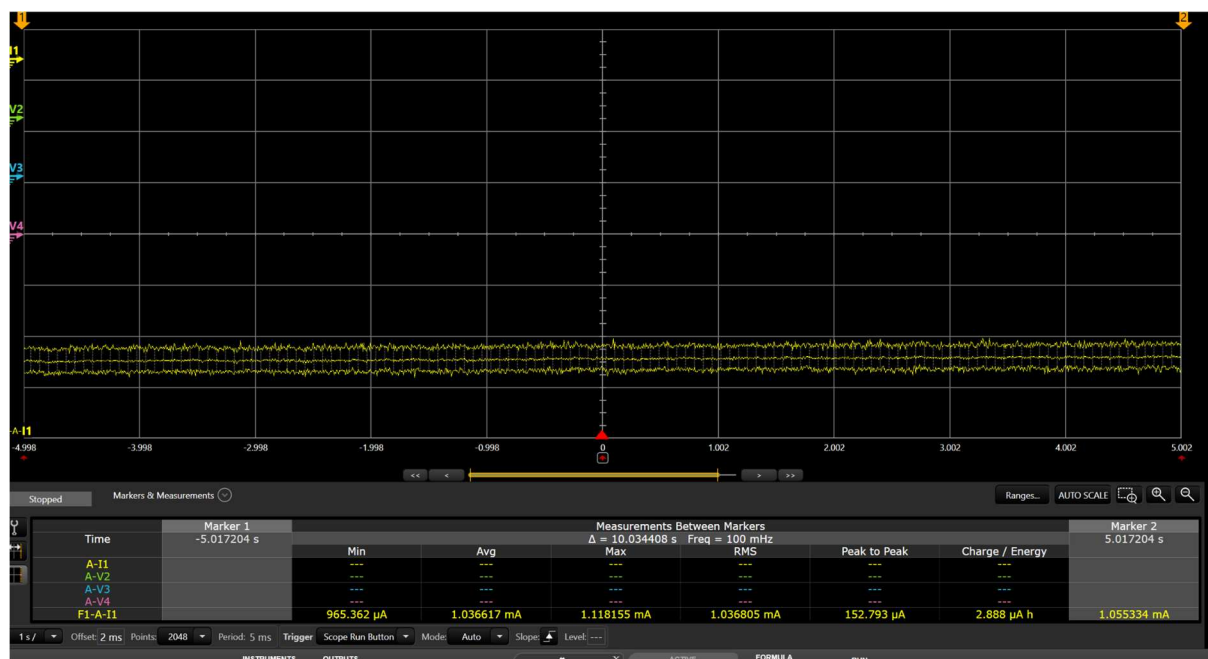


Figure 6: Timer Implementation

The measurement of the full timer implementation in Figure 6 shows a result nearly identical to the loop variant in Figure 7, though it shows a slightly higher average efficiency. It should be noted however, that repeated test series also produced reversed results, where the loop had a slight advantage. These differences are of an extremely small magnitude. Notably, this solution, with over 150 lines of code, is significantly more complex than the simple wait loop. In the loop measurement, more pronounced noise is visible, the exact cause of which could not be determined within the scope of this experiment.

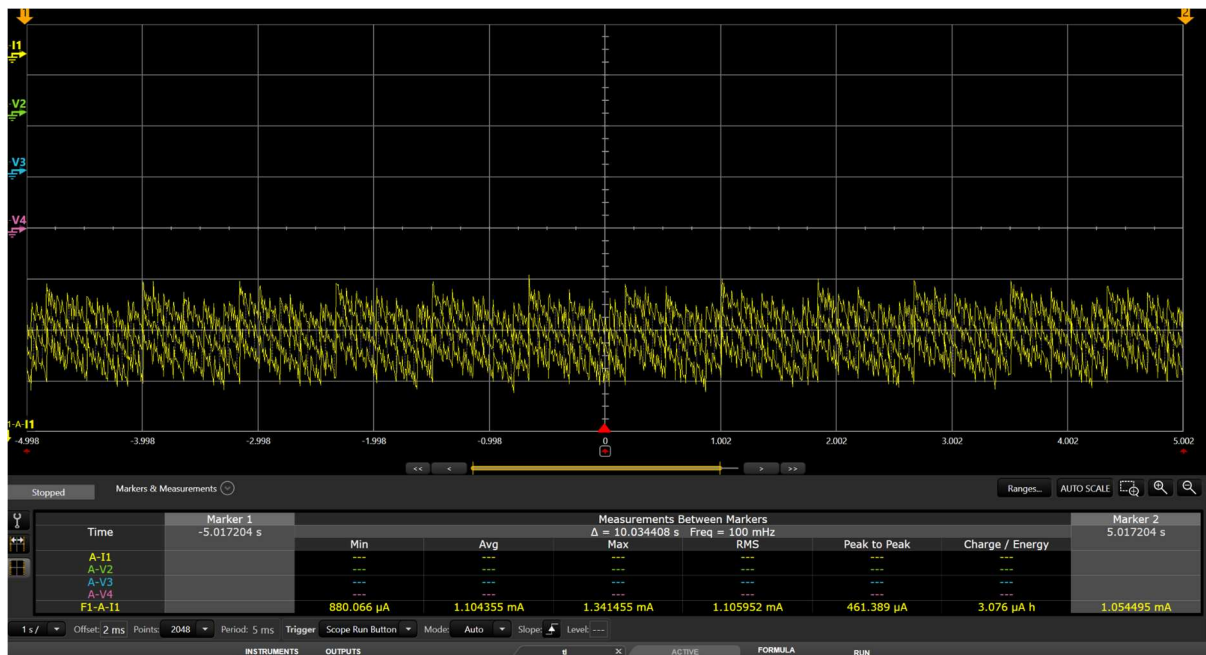


Figure 7: "Timer" Loop

Discussion

Regarding energy efficiency, both approaches perform at a nearly identical level for this specific task. However, the timer implementation requires deeper hardware knowledge, as it requires manual configuration of the peripheral clock via the RCC and synchronization of hardware registers like prescaler and auto-reload register. Additionally, managing the NVIC and manually clearing status flags within an Interrupt Service Routine is essential to avoid infinite execution loops. Ultimately, the decisive advantage of the hardware timer lies less in its energy consumption and primarily in its ability to free up the CPU for parallel tasks.

Data Processing Pipeline (with and without debug logging)

This test case evaluates the impact of debug logging on the execution efficiency of a data processing pipeline. The implemented test case simulates a typical embedded workload, where an array of input values is processed sequentially. Two configurations were compared: one with debug logging enabled via a preprocessor directive (`#define DEBUG_LOG`), and one with logging completely disabled. This first setup reflects a realistic scenario in which verbose debug output is accidentally left active in a production environment.


```

#define DEBUG_LOG
#ifdef DEBUG_LOG
    FILE *log_file;
    #define LOG(fmt, ...) fprintf(log_file, fmt, __VA_ARGS__)
    #define PREFIX "LOG_ENABLED"
#else
    #define LOG(fmt, ...)
    #define PREFIX "LOG_DISABLED"
#endif

```

```

void run() {
    // Main processing loop
    for (int i = 0; i < DATA_SIZE; i++) {
        float result = process_value(input[i]);
        output[i] = result;

        // Debug logging inside loop
        LOG("Index %d: input=%.2f output=%.2f\n", i, input[i], result);
    }
}

```

The results show a drastic difference in execution time between the two configurations. With logging enabled, the measured runtime is approximately 0.286 seconds, with small fluctuations of about ± 0.002 seconds between runs. In contrast, the version without logging completes in approximately 0.003 seconds and remains highly stable across repeated measurements. This corresponds to a slowdown by nearly two orders of magnitude when debug output is active.

The observed overhead can be attributed primarily to the cost of file I/O and string formatting. Each call to `fprintf` introduces significant additional work, including formatting floating-point values, handling arguments, and interacting with the underlying file system. In embedded systems, such operations are particularly expensive due to limited resources and potentially slow peripheral communication.

Discussion

Although no direct energy measurements were performed in this test, the strong increase in execution time suggests a corresponding rise in energy consumption. A longer active CPU time generally implies higher overall energy usage, especially when the processor cannot enter low-power states during blocking I/O operations. Therefore, it is reasonable to assume that the logging-enabled configuration is significantly less energy-efficient.

Algorithm efficiency

This test case evaluates the runtime difference between a slow algorithm and a fast algorithm. Bubble Sort was implemented for the slow algorithm and for the fast algorithm Quick Sort was used. Bubble Sort has a complexity of $O(n^2)$, while Quick Sort has a complexity of $O(n \cdot \log(n))$. Due to the well-known complexities, it is expected that Quick Sort will be faster than Bubble Sort.

A significant difference in execution time can be observed in the runtime comparison between Bubble Sort and Quick Sort. Both experiments involved sorting 100'000 arrays. For an array size of 10'000 elements, Bubble Sort required 27.4893 seconds, while Quick Sort completed the same task in just 0.1552 seconds. This demonstrates a significant difference in execution time can be observed in the runtime comparison between Bubble Sort and Quick Sort. Quick Sort had a considerably higher efficiency when processing large datasets. A similar trend can be seen for smaller input sizes. With a reduced dataset size of just 50 elements in an array, Bubble Sort took 0.3806 seconds to complete the task, whereas Quick Sort still performed faster at approximately 0.1165 seconds.

Discussion

Although the difference is less pronounced for smaller arrays, the measurements clearly indicate that Quick Sort scales much more efficiently as the dataset size increases. This is the expected outcome, owing to the different complexities of the algorithms. Due to the difference of the runtimes, the Quick Sort algorithm enables the CPU to finish the task more quickly, allowing it to start another task earlier or enter an idle state. This saves energy and is more sustainable.

Conclusion

The extremely low energy footprint of the system made it difficult to capture absolute values, as the signal-to-noise ratio was too low for the available equipment. While these measurements provide an indication of efficiency trends, they should be viewed as relative rather than absolute. Achieving higher precision would require a more specialized measurement setup and a suitable environment to perform these measurements.

The results show that peripheral and I/O operations are more resource-intensive than code optimizations. Consequently, optimization should focus on reducing runtime complexity to return the CPU to idle or other tasks quickly, while minimizing peripheral and I/O usage.

In conclusion, the data highlights that even in resource-constrained environments, software remains the deciding factor for system longevity. Optimization, which focuses on reducing active CPU time and streamlining I/O, is essential to bridge the gap between advanced hardware capabilities and real-world energy efficiency.

List of Figures

Figure 1: CT Board [Online].URL: https://ennis.zhaw.ch/wiki/doku.php [accessed:5.5.2026]....	2
Figure 2: Timer with LED.....	3
Figure 3: Busy Loop.....	4
Figure 4: Normal Interrupt.....	5
Figure 5: Sleep.....	5
Figure 6: Timer Implementation.....	6
Figure 7: "Timer" Loop.....	7

Appendix

ASE2 Case-Study Repo:

https://github.zhaw.ch/steinse1/ASE2_Case-Study

Ennis Dokumentation for the full specifications of the CT board (device used in this case study):

<https://ennis.zhaw.ch/wiki/doku.php>